

Operating System #4. Deadlock & Semaphore.

1. Core Map.

- Lock/Mutex: protect critical section
- Deadlock: Permanent waiting cycle,
- Semaphore: Counter-based access control.

"Synchronization prevents race condition, but incorrect synchronization can create deadlock."

2. Resource Waiting

(Def) Resource: anything that an execution flow needs before it can continue

<Example>

lock or mutex, memory area, file handle, database row, socket, I/O device, buffer slot, connection pool slot.

3. Deadlock.

3.1. Def: a set of processes or threads are waiting for each other forever.

<Example>. Thread A holds lock A and waits for lock B
(Thread B = B // A.)

Both are not able to continue ✓

3.2. 4 necessary conditions. that create deadlock.

① Mutual Exclusion.

↳ at least one resource cannot be shared.

② Hold and Wait.

↳ A thread holds one resource while waiting for another.

③ No preemption

↳ The resource cannot be forcibly taken away safely

④ Circular wait.

↳ A waiting cycle exist between threads/resources.

3.3. Deadlock prevention.

① global lock order: always acquire multiple locks in the same order.

② Short lock scope: release the lock as soon as the shared-state operation is done.

③ avoid nested lock: do not hold one lock while asking for another unless necessary.

④ Try-lock/time out: do not wait forever; give up and retry

⑤ No slow work inside lock.

: do not wait for network, disk, or user input while holding a lock

3.4 Handling Strategies.

① Prevention — Def: design so at least one deadlock condition cannot hold.
— Common use: Lock ordering, short critical sections.

② Avoidance — Def: grant resources only if the future state is safe.
— Common use: Systems with known maximum resource needs.

③ Detection + recovery
— Def: Find deadlock after it happens, then abort, rollback or release resources.
— Commonly used: Databases, transaction systems

~~④ Ignore~~

④ Ignore / restart — Accept race deadlocks and recover externally
— Commonly used: Some general-purpose systems at OS level.

3.5. Deadlock vs Starvation vs Livelock

<Deadlock>

- Def: Threads wait for each other in a cycle.
- No progress.

<Starvation>

- Def: One thread keeps losing access while others continue.
- System progresses, one thread may not.

<Live lock>.

- Def: Thread keep reacting to each other but fail to complete work.
- Movement exists, useful progress does not ✓

4. Semaphore.

4.1. Def.

"Synchronization object based on a non-negative counter,"

- Semaphore counter = number of available permits.
- wait/acquire/P/down = Take one permit; count decreases by 1
- signal/release/V/up/post = Return one permit; count increases by 1 or wakes a waiter.

4.2. Binary vs Counting Semaphore.

<Type 1: Binary semaphore>

- Count: 0 or 1.
- Main use: Gate like access control; similar to a lock in ~~so~~ simple cases

<Type 2: Counting semaphore>

- Count: 0 or N
- Main use: Allows up to N users of a limited resource.

<Example>.

- semaphore(1) $\rightarrow$ at most 1 thread can pass.
- semaphore(3) $\rightarrow$ at most 3 threads pass.

4.3. Mutex vs Semaphore.

	< Mutex >	< Semaphore >
Main Idea.	Ownership-based mutual exclusion	Counter-based on permit control
Best for	Protecting one critical section	Set of permit Limiting resource count or signaling
Count	Usually one owner	0 to N permit
Ownership	The locker should unlock	Post/signal may be used for coordination patterns
Common mistake	Holding it too long	Using it as a vague replacement for every lock.

< key point > ,

" use a mutex when the question is 'Who may touch this shared data now?' "

" use a semaphore when the question is 'How many permits / resources are available' "

5. Essential Patterns.

5.1. Protecting shared data.

```
lock(mutex);  
update_shared_data();  
unlock(mutex);
```

- The whole logical operation must be protected together.
- Do not protect only half of a check-update sequence.

5.2. Limiting access to N identical resources.

```
slots = semaphore(N);  
wait(slots);  
use_one_resource();  
signal(slots);
```

- This is useful when several equivalent resources exist, such as connection slots, worker slots, or buffer space.

5.3. Producer-Consumer structure.

```
empty_slots = Semaphore(buffer_size);  
full_slots = Semaphore(0);  
mutex = Mutex();
```

<Producer>

```
wait(empty_slots);  
lock(mutex);  
push_item();  
unlock(mutex);  
signal(full_slots);
```

<Consumer>

```
wait(full_slots);  
lock(mutex);  
e ← pop - item();  
pop unlock(mutex);  
signal(empty_slots);
```

⇒ Semaphores count availability.
(The mutex protects the queue structure.)

6. Common mistake

6.1. Different lock order

- $\ominus$: creates circular wait
- How to fix: Define one global order.

6.2. Holding lock while waiting

- $\ominus$: Blocks others while the current thread is blocked.
- How to fix: Release before slow or uncertain waits.

6.3. Forget unlock/signal

- $\ominus$: Other threads may wait forever.
- How to fix: Use finally, defer or RAII.

6.4. lock too much.

- $\ominus$: Correctness may hold, but concurrency collapses
- How to fix: keep critical section small.

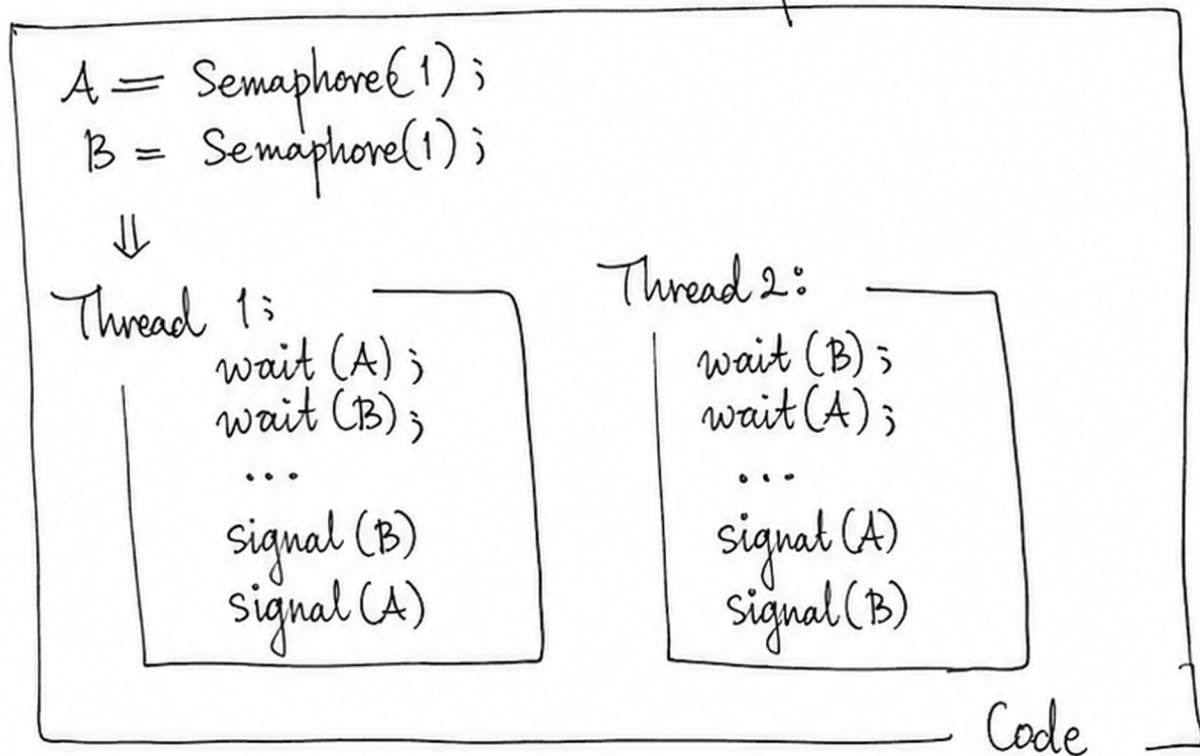
6.5. lock too late.

- $\ominus$: Earlier check may already be stale.
- How to fix: Protect the full logical operation

6.6. Semaphore without mutex.

- $\ominus$: Permit count may be correct while shared structure is corrupted.
- How to fix: Use mutex for shared data structure updates.

7. Dead lock with Semaphore.



"Semaphore is not magic. It is still a resource, Wrong ordering can still create a waiting cycle."

8. What to memorize.

- Deadlock is permanent waiting cycle.
- 4 conditions of Deadlock: mutual exclusion, hold and wait, no preemption, and circular wait.
- The most practical prevention is "Consistent resource ordering"
- Mutex protects critical section with an owner at a time.
- A semaphore controls permits with a counter.
- Counting semaphores are useful for limited capacity, not just mutual exclusion
- Protect shared mutable state, but keep the protected area small.
- Always release what you acquire.